Clean Architecture: usporedba primjera i Kuharice

Uspoređeno: profesorov primjer **TW1_P05_Online_EFDemo** (zip) i trenutni **130926_Kuharica_Project** na disku (stanje 22.10.2026).

Ukratko: Kuharica funkcionalno pokriva većinu onoga što primjer radi, a u nekim stvarima je i ispred (Serilog, StudentPredmet endpointi, ErrorResponse s TraceId, Tests projekt). Najveća razlika nije u funkcionalnosti nego u **rasporedu slojeva i smjeru referenci**: primjer ima zaseban Domain projekt i Application sloj koji ne ovisi o bazi, a kod tebe BLL ovisi o DAL-u.

---

1. Struktura projekata (najvažnija razlika)

Primjer	Kuharica	Napomena
`*.Domain`	❌ ne postoji	Entiteti + sučelja repozitorija. Ne referencira **ništa**.
`*.Application`	`BusinessLayerLogic`	Servisi, DTO-i, mapiranje, validatori, iznimke, auth logika
`*.DAL`	`130926_Kuharica_Project.DAL`	DbContext, konfiguracije, migracije, implementacije repozitorija
`*.API`	`130926_Kuharica_Project`	Kontroleri, middleware, Program.cs, JWT ekstenzija
❌ nema	`Tests`	Primjer nema testove

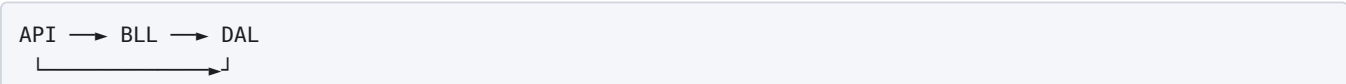
Smjer referenci

Primjer (Clean Architecture):



- `Domain` ne zna ni za koga.
- `Application` zna **samo** za `Domain` (ne zna da postoji EF Core ni SQL Server).
- `DAL` zna samo za `Domain` i **implementira** sučelja koja su tamo definirana.
- `API` spaja sve (jedini zna za oba, zbog DI registracije).

Kuharica (klasična N-slojna):



- Entiteti (`Model/`) i sučelja repozitorija (`Interface/`) su u DAL-u.
- BLL ovisi o DAL-u, tj. poslovna logika ovisi o bazi. To je upravo ono što Clean Architecture okreće (Dependency Inversion): sučelje repozitorija treba "pripadati" sloju koji ga koristi, a ne sloju koji ga implementira.

Što fali: Domain projekt; premještanje `Model/` i `Interface/` iz DAL-a u Domain; okretanje referenci (BLL → Domain umjesto BLL → DAL; DAL → Domain).

---

2. Domain sloj (entiteti)

	Primjer	Kuharica
Lokacija	`Domain/Entities`	`DAL/Model`
Bazna klasa	`abstract class BaseEntity { int Id }`	❌ svaki entitet ima svoj `Id`

	Primjer	Kuharica
Navigacijske kolekcije	`ICollection<..> = new List<..>()` (nikad null)	`ICollection<..>?` (nullable, zato trebaš `!` u `ThenInclude`)
Reference u M:N tablici	`Student Student { get; set; } = null!;`	`Student? Student`
Nazivi FK	`StudentId`, `PredmetId`	`StudentID`, `PredmetID`
User / Role / UserRole	✅ entiteti u bazi	❌ (korisnici su hardkodirani u AuthControlleru)

Nazive (`Name` vs `Naziv`, `ID` vs `Id`) ne moraš mijenjati da bi arhitektura bila "clean", to je samo stil. `BaseEntity` je bitan jer o njemu ovisi generički repozitorij (`where T : BaseEntity`).

---

3. DAL sloj

	Primjer	Kuharica
DbContext	`Data/TW1P05DBContext.cs`	`AppDbContext.cs` (u rootu)
Konfiguracija modela	Zasebna klasa po entitetu: `Data/Configurations/*Configuration.cs` (`IEntityTypeConfiguration<T>`) + `ApplyConfigurationsFromAssembly`	Sve inline u `OnModelCreating`
Ograničenja kolona	`IsRequired()`, `HasMaxLength(100)`, `ToTable(...)`, `HasIndex(...).IsUnique()`	❌ nema (stringovi su `nvarchar(max)`, nullable)
Cascade delete	Eksplicitno `OnDelete(DeleteBehavior.Cascade)`	Implicitno (EF default)
Seed podataka	`HasData` za uloge Admin/User	❌
Generički repozitorij	`Repository<T> : IRepository<T>` (zajednički CRUD jednom napisan)	❌ svaki repo piše sve sam (`ICrud<T>` je samo sučelje)
Specifični repozitoriji	`StudentRepository : Repository<Student>, IStudentRepository` + dodatne metode (`GetStudentWithPredmetiAsync`)	`StudentRepository : ICrud<Student>`
`SaveChanges`	**Nije** u repozitoriju, nego u **Unit of Work**	U svakoj metodi repozitorija
Unit of Work	✅ `IUnitOfWork` (Domain) + `UnitOfWork` (DAL): drži `Studenti`, `Predmeti`, `Users` + `SaveChangesAsync()`	❌ ne postoji
Update	Servis dohvati entitet, promijeni polje, `UpdateAsync` + `SaveChangesAsync`	Repo radi fetch-then-update s `(id, entity)`
DI registracija	`Extensions/DALServiceExtensions.AddDALServices(connString)`	Sve u `Program.cs`

Što fali: `IEntityTypeConfiguration` klase, `BaseEntity` + generički `Repository<T>`, Unit of Work, extension metoda za registraciju.

Napomena: dodavanje `HasMaxLength`/`IsRequired` mijenja shemu, pa će trebati nova migracija. Samo premještanje entiteta u drugi projekt/namespace **ne** mijenja shemu, postojeće migracije ostaju valjane (snapshot će dobiti nove namespaceove pri sljedećoj migraciji).

---

4. Application sloj (tvoj BLL)

	Primjer	Kuharica
Ime projekta	`Application`	`BusinessLayerLogic`
Referencira	samo `Domain`	`DAL`
Servisi primaju/vraćaju	**DTO-e** (`CreateStudentDto` → `StudentDto`)	**Entitete** (`Student`)

	Primjer	Kuharica
Servisi rade preko	`IUnitOfWork`	`ICrud<T>` direktno
Update/Delete vraćaju	`bool` (kontroler iz `false` radi 404)	entitet ili `null`
`Exists` metode	✅ `StudentExistsAsync`	❌
DTO-i	`Application/DTOs/Student/...` kao **record**: `StudentDto`, `CreateStudentDto`, `UpdateStudentDto`, `StudentWithPredmetiDto`	`API/DTO's/` kao klase
Mapiranje	Ručne extension metode `Mapping/MappingExtensions.cs` (`student.ToDto()`, `dto.ToEntity()`)	AutoMapper `MappingProfile` u API-ju, poziva se u kontroleru
Validacija	**FluentValidation** (`Validators/*Validator.cs`)	DataAnnotations (`[Required]`, `[MaxLength]`) na DTO-ima
Iznimke	`Exceptions/` (Authentication, UserAlreadyExists, InvalidRefreshToken)	`Exceptions/` (NotFound, Conflict) ✅ isti princip
Options klasa	`Options/JwtSettings` (+ `IOptions<JwtSettings>`)	❌ čita `_config["Jwt:Key"]` direktno
Auth servisi	`AuthService`, `TokenService`, `PasswordHasher` (+ sučelja)	`TokenService` u **API**/Auth
DI registracija	`Extensions/ApplicationServiceExtensions.AddApplicationServices(config)`	U `Program.cs`

Što fali: DTO-i i mapiranje premješteni u Application, servisi koji vraćaju DTO-e (kontroler više ne mapira), FluentValidation, `JwtSettings` + IOptions, auth logika iz API-ja u Application, extension metoda za registraciju.

AutoMapper možeš zadržati (samo `MappingProfile` seli u Application i `AddAutoMapper` pokazuje na taj assembly). Ručno mapiranje iz primjera je jednostavnije i ne treba paket, pa je to tvoja odluka.

Pazi kod FluentValidation: u primjeru su validatori registrirani (`AddValidatorsFromAssembly`), ali ih **nitko ne poziva**: API nema `FluentValidation.AspNetCore` auto-validaciju, a servisi ne injektiraju `IValidator<T>`. Znači da u primjeru validacija zapravo ne radi (osim što `[ApiController]` vrati 400 za null tijelo). Ako to budeš uvodio, treba ili ručno pozvati `validator.ValidateAsync(dto)` u servisu (i baciti iznimku koju middleware mapira u 400) ili dodati auto-validaciju.

---

5. API sloj

	Primjer	Kuharica
Kontroleri ovise o	servisima (koji vraćaju DTO)	servisima + `IMapper`
Nazivi	`StudentsController`, `PredmetiController` (množina)	`StudentController`, `PredmetController`
Async + `CancellationToken`	✅ svugdje	❌ sinkrono (dogovorena odluka ranije)
`[ProducesResponseType]`	✅ na svakoj akciji (bolji Swagger)	❌
PUT	provjera `id != dto.Id` → 400, vraća **204 NoContent**	vraća 200 + DTO
DELETE	**204 NoContent**	200 + obrisani DTO
Dodatni endpointi	`GET /api/students/{id}/predmeti`, `GET /api/predmeti/{id}/studenti`	❌ (ali imaš cijeli `StudentPredmetController`)
Upis studenta na predmet	❌ **primjer to nema**	✅ `POST/DELETE /api/StudentPredmet`

	Primjer	Kuharica
Autorizacija	GET anonimno; POST/PUT `User,Admin`; DELETE `Admin`	Sve traži login; POST/PUT/DELETE `Admin`
Auth endpointi	`register`, `login`, `refresh`, `logout`	samo `login` (fiksni admin/korisnik)
Middleware	`ExceptionHandlingMiddleware` + `UseExceptionHandling()` extension	`GlobalExceptionMiddleware` + `ErrorResponse` (s TraceId) ✅
JWT konfiguracija	`Extensions/JwtServiceExtensions.AddJwtAuthentication(config)`	Inline u `Program.cs`
Program.cs	~70 linija, samo poziva `AddDALServices`, `AddApplicationServices`, `AddJwtAuthentication`	~110 linija, sve registracije ručno
Logiranje	ugrađeni `ILogger`	**Serilog** (konzola + datoteka) ✅
.NET	net10.0	net8.0 (ne treba mijenjati)

6. Što Kuharica ima, a primjer nema

- **Serilog** + `UseSerilogRequestLogging` (zahtjev projektnog zadatka).
- **StudentPredmet** CRUD i servisna provjera (404 ako student/predmet ne postoji, 409 za dupli upis).
- **ErrorResponse** s `TraceId`: konzistentniji error odgovor od anonimnog objekta u primjeru.
- **Swagger Bearer** kao `SecuritySchemeType.Http` (lijepiš samo token, bez "Bearer ").
- **Tests** projekt (još prazan).

Sve ovo ostaje i samo se seli u odgovarajući sloj.

7. Što uopće nije implementirano u Kuharici (popis)

1. Domain projekt (entiteti + sučelja repozitorija + `IUnitOfWork`).
2. `BaseEntity`.
3. Generički `IRepository<T>` / `Repository<T>` + specifična sučelja (`IStudentRepository` s dodatnim metodama).
4. Unit of Work (`SaveChanges` izvan repozitorija).
5. `IEntityTypeConfiguration` klase + `MaxLength` / `IsRequired` / `ToTable`.
6. DTO-i i mapiranje u Application sloju; servisi vraćaju DTO-e.
7. Create/Update/"With" DTO-i po entitetu (`StudentWithPredmetiDto` i endpoint `{id}/predmeti`).
8. FluentValidation.
9. `JwtSettings` options klasa (IOptions umjesto `_config["..."]`).
10. Korisnici u bazi: `User`, `Role`, `UserRole`, seed uloga, `PasswordHasher` (PBKDF2), `register`.
11. Refresh token + `logout` (revoke).
12. Extension metode za DI (`AddDALServices`, `AddApplicationServices`, `AddJwtAuthentication`, `UseExceptionHandling`).
13. `[ProducesResponseType]` na akcijama; PUT/DELETE vraćaju 204.
14. Async/await + `CancellationToken` (namjerno izostavljeno dosad).

Točke 10, 11 i 14 su "veće" i nisu strogo dio Clean Architecture, nego dodatne funkcionalnosti primjera. Predlažem da ih odlučimo posebno.

8. Sitnice koje sam usput našao u Kuharici

- `Program.cs`: `app.UseAuthorization();` je napisan **dvaput**.
- `StudentPredmetController.GetByID` ima povratni tip `ActionResult<StudentPredmet>` (entitet), a vraća `StudentPredmetDto`. Swagger zato prikazuje krivu shemu.

- `IPredmetService.Create` vraća `Predmet?`, a repozitorij nikad ne vraća null (može biti `Predmet`).
- `Student.Name` je `required string`, a `StudentDto` / `StudentUpdateDto` imaju `string?`. `StudentCreateDto` ima `[MaxLength(30)]`, a `StudentUpdateDto` nema, pa update može spremiti dulje ime.
- `PredmetRepository.GetAll/GetById` rade `Include` studenata, ali `PredmetDto` ih ne prikazuje (nepotreban JOIN; u primjeru je to riješeno zasebnom metodom `GetPredmetWithStudentimaAsync`).
- `ReferenceHandler.IgnoreCycles` vjerojatno više nije potreban jer kontroleri vraćaju DTO-e, a ne entitete.
- Neiskorišteni `using`-i u kontrolerima (`DAL`, `DAL.Repo`, `EntityFrameworkCore`, `System.Reflection.Metadata.Ecma335`...). Kad kontroleri više ne budu smjeli znati za DAL, to će se ionako počistiti.
- `Tests` projekt referencira API, ali još nema `Moq` paket ni pravi test.

---

9. Predloženi redoslijed prepravka (pregled, bez koraka)

1. Napraviti **Domain** projekt, preseliti entitete (+ `BaseEntity`) i sučelja repozitorija; okrenuti reference (DAL → Domain, BLL → Domain).
2. Preimenovati/organizirati **BLL u Application** (može ostati ime `BusinessLayerLogic` ako ti je lakše), izbaciti referencu na DAL.
3. Preseliti **DTO-e i mapiranje** u Application; servisi primaju/vraćaju DTO-e, kontroleri gube `IMapper`.
4. **Entity konfiguracije** u DAL-u (+ nova migracija za MaxLength/Required).
5. **Generički repozitorij + Unit of Work**, `SaveChanges` izlazi iz repozitorija.
6. **Extension metode za DI**, očistiti `Program.cs`.
7. Kontroleri: `[ProducesResponseType]`, 204 za PUT/DELETE, `{id}/predmeti` endpointi.
8. Po želji: FluentValidation, `JwtSettings`, korisnici u bazi + hash lozinke + refresh token, async.
9. Tek onda **unit testovi** (mockaju se servisi iz Application sloja, pa ih ima smisla pisati nakon prepravka).

Svaki korak završava buildom i brzim Swagger testom, da se greške ne gomilaju.